



Simple linear time algorithm for sorting strings in omega-order with applications

Ruixi Luo^{1,2} · Taikun Zhu^{1,2} · Kai Jin^{1,2}

Received: 14 January 2025 / Accepted: 27 March 2026
© The Author(s) 2026

Abstract

We present a simple linear time algorithm for the following sorting problem: Given n words $A_1, \dots, A_n$, find a permutation $\{\pi_1, \dots, \pi_n\}$ of $1, \dots, n$ so that $(A_{\pi_1})^\omega \leq \dots \leq (A_{\pi_n})^\omega$, where $(A)^\omega$ denotes the infinitely repeating string $AAA\dots$. Let L denote the total length of the given strings. We note that the running time of our algorithm is $O(L)$ even if the size of alphabet is beyond $O(1)$. We also present an $O(L + n \log n)$ time algorithm for the restricted model where we are only allowed to compare symbols. In other words, the main result of this paper is that the time complexity of sorting $A_1, \dots, A_n$ in omega-order is the same as that of sorting them in lexicographic order under all common models (bounded alphabet, unbounded alphabet, compare-based model). Our main results find applications in related problems such as rearranging and concatenating given words so that the result is lexicographically smallest or largest.

Ruixi Luo and Taikun Zhu have contributed equally to this work

This paper is an extended version of our paper published in the 3rd International Joint Conference on Theoretical Computer Science - Frontier of Algorithmic Wisdom (IJTCS-FAW 2022).

✉ Kai Jin
cscjkk@gmail.com

Ruixi Luo
luorx@mail2.sysu.edu.cn

Taikun Zhu
zhutk3@mail2.sysu.edu.cn

¹ School of Intelligent Systems Engineering, Shenzhen Campus of Sun Yat-sen University, No.66, Gongchang Road, Guangming District, Shenzhen 518107, Guangdong, China

² Guangdong Provincial Key Laboratory of Fire Science and Intelligent Emergency Technology, Guangzhou 510006, Guangdong, China

1 Introduction

Given n non-empty words $A_1, \dots, A_n$ over an ordered alphabet Σ of size σ , we consider how to rearrange and concatenate $A_1, \dots, A_n$ so that the obtained string S is lexicographically smallest (or largest, respectively). Let A^ω denote $AA \dots$, which we call the *repeating string* of A . We show that the smallest outcome of S happens when we arrange the words so that their repeating strings are in increasing (or non-decreasing, to be precise) order, and the largest outcome of S happens when we arrange the words so that their repeating strings are in decreasing (or, non-increasing, to be precise) order. In other words, assuming without loss of generality that $A_1^\omega \leq \dots \leq A_n^\omega$, it holds that $A_1 \dots A_n \leq S \leq A_n \dots A_1$ for any S produced by concatenation (Theorem 4).

According to the above result (which we call *string rearrangement inequalities*), our task is as follows.

Problem 1. Given non-empty words $A_1, \dots, A_n$, find a permutation $(\pi_1, \dots, \pi_n)$ of $\{1, \dots, n\}$ so that $A_{\pi_1}^\omega \leq \dots \leq A_{\pi_n}^\omega$.

This is different from the classic problem of sorting $A_1, \dots, A_n$ in lexicographic order. For example, consider $A_1 = \text{“ab”}$, $A_2 = \text{“aba”}$, and $A_3 = \text{“abc”}$. We have $A_1 < A_2 < A_3$, whereas $A_2^\omega < A_1^\omega < A_3^\omega$.

Let L denote the total length of the given words; namely, $L = |A_1| + \dots + |A_n|$, where $|A|$ denotes the length of string A . We present three algorithms for finding the mentioned rearrangement, beating the trivial $\Omega(L \log n)$ time bound of plain algorithms (the details are presented after our Algorithm 1). (1) We first show an $O(\sigma L)$ time algorithm, which is optimal when σ is assumed to be $O(1)$. (2) We then show an $O(L + \sigma)$ time algorithm for

integer alphabets. The term $O(L + \sigma)$ can be regarded as $O(L)$ for linearly sortable alpha-

bets (i.e. $\sigma = L^{O(1)}$). (3) Moreover, we show an $O(L + n \log n)$ time algorithm based on Three-Way Radix Quicksort [1]. This last algorithm is designed for the comparison based model where we are allowed to compare two symbols in Σ but not read or manipulate a single symbol. In each case, our running time complexity for sorting $A_1^\omega, \dots, A_n^\omega$ are the same as the best running time complexity for sorting $A_1, \dots, A_n$ in lexicographic order.

Technically, our algorithms are mainly built upon two lemmas. First, sorting $A_1^\omega, \dots, A_n^\omega$ is equivalent to sorting $M_1, \dots, M_n$ (see Lemma 5), where M_i is a certain prefix of A_i^ω that can be computed from $A_1, \dots, A_n$; see the definition in (5). Second, the total length of $M_1, \dots, M_n$ is bounded by $O(L)$ (see Lemma 6). Our algorithms work by computing $M_1, \dots, M_n$ and then sorting them. The choice of M_i 's is subtle and is the most challenging part in our approach.

Outline. We present the sorting algorithms in section 2 and present the string rearrangement inequalities in section 3. In section 4, we implement the inequalities and algorithms to solve some variants of the original problem.

1.1 Related work

Combinatorics on words (MSC: 68R15) maintains strong connections to diverse fields of mathematics and has found significant applications in theoretical computer science and molecular biology, particularly in DNA sequence analysis [2–7]. Primitive words over an alphabet Σ have received considerable attention due to their applications in formal language theory and the algebraic theory of codes [8–11].

It is worth noting the longstanding conjecture that the language of primitive words is not context-free [8–10, 12]. To the best of our knowledge, this conjecture remains open. In contrast, Berstel and Boasson [12] established that the language of Lyndon words is not context-free.

Fredricksen and Maiorana [13] demonstrated that concatenating all Lyndon words whose lengths divide a given integer n in lexicographic order yields a de Bruijn sequence. Subsequently, Au [14] showed that replacing the condition “dividing n ” with “identical to n ” produces a sequence that contains every primitive word of length n exactly once as a factor.

Dolce et al. [15] introduced the concept of infinite order (ω -order), demonstrating that while lexicographic order lacks certain desirable properties for specific problems, ω -order satisfies these requirements.

The ω -order has been notably applied in the extended Burrows-Wheeler Transform (eBWT) [16]. Mantaci et al. [16] briefly mentioned that employing the generalized suffix array [17] could reduce the time complexity of sorting words in ω -order to $O(L)$; however, their paper provided neither proof nor detailed explanation. Several years later, Nong et al. [18] presented the first construction algorithm for eBWT, requiring $O(L \log L)$ time and $O(L \log \sigma)$ bits of working space. More recently, by adapting the induced sorting algorithm of Nong et al. [19], Boucher et al. [20] (for extended BWT) and Bannai et al. [21] (for bijective BWT) achieved the $O(L)$ time bound for sorting all conjugates of the given words in ω -order under linearly sortable alphabets using similar procedures.

Nevertheless, these linear-time methods [20, 21], which rely on generalized suffix arrays and recursive procedures, appear more complicated than our approach. This complexity is understandable given that they sort all conjugates rather than just the words themselves. Our results demonstrate that sorting the given words in ω -order can be accomplished through a simpler deterministic string sorting algorithm. We will elaborate on the distinctions between these approaches in Subsection 2.3 after presenting our main results.

1.2 Preliminaries

Definition 1 *The n -th power of word A is defined as:*

$$A^n = \begin{cases} AA^{n-1}, & n > 0; \\ \text{empty word}, & n = 0. \end{cases}$$

A word A is **non-primitive** if it equals B^k for some word B and integer $k \geq 2$. Otherwise, A is **primitive**. (By this definition the empty word is not primitive.)

The next lemma summarizes three results about the powers proved by Lyndon and Schützenberger [22]; see their Lemmas 3 and 4, and Corollary 4.1. (More introductions of these results can be found in section 1.3 of [3].)

Lemma 1 [22] *Given words A and B , there exist C, k, l such that $A = C^k$ and $B = C^l$ when one of the following conditions holds ($m_1, m_2 \geq 1$ are integers.):*

1. $AB = BA$.
2. Two powers A^{m_1} and B^{m_2} have a common prefix of length $|A| + |B|$
3. $A^{m_1} = B^{m_2}$.

Definition 2 The **root** of a (non-empty) word A , denoted by $\text{root}(A)$, is the unique primitive word B such that A is a power of B . The uniqueness of root is obvious, a formal proof can be found in Corollary 4.2 of [22] or in [11].

The following lemma should be well-known, and its simple proof can be found in the appendix.

Lemma 2 *Assume A is a non-empty word. Find the largest $j < |A|$ such that the prefix of A with length j equals the suffix of A with length j . Let $k = |A| - j > 0$. Then,*

$$|\text{root}(A)| = \begin{cases} k, & |A| \equiv 0 \pmod{k}; \\ |A|, & |A| \not\equiv 0 \pmod{k}. \end{cases} \quad (1)$$

Before the next lemma, we first specify the definition of lexicographic order for repeating strings.

Definition 3 Let $A = a_1 a_2 \cdots a_p$ and $B = b_1 b_2 \cdots b_q$ be non-empty finite words. The repeat-

ing strings A^ω and B^ω are lexicographically ordered as follows: $A^\omega < B^\omega$ ($A^\omega > B^\omega$) if there exists $k \geq 1$ such that the k -th character of A^ω is smaller (larger) than the k -th character of B^ω , and all previous characters are equal. $A^\omega = B^\omega$ if there is no such k .

The following lemma is fundamental to our algorithm.

Lemma 3 (Corollary 11 of [15]) *For non-empty words A and B , the relation between A^ω and B^ω is the same as the relation between AB and BA . In other words,*

$$A^\omega < B^\omega \Leftrightarrow AB < BA, \quad (2)$$

$$A^\omega > B^\omega \Leftrightarrow AB > BA. \quad (3)$$

$$A^\omega = B^\omega \Leftrightarrow AB = BA, \quad (4)$$

See a proof of Lemma 3 in Appendix [Appendix B](#). As a corollary, “ $AB \leq BA$ and $BC \leq CB$ implies $AC \leq CA$ ”. This transitivity is not obvious without Lemma 3.

2 Sorting repeating words

This section presents a linear time algorithm for sorting the repeating words $A_1^\omega, \dots, A_n^\omega$.

We first assume that $\sigma = O(1)$ and then discuss the more challenging case where σ is arbitrary in subsection 2.1 and 2.2.

Theorem 1 $A_1^\omega \cdots A_n^\omega$ can be sorted in $O(L) = O(\sum_i |A_i|)$ time for the case $\sigma = O(1)$.

Because $A_i^\omega = \text{root}(A_i)^\omega$, sorting $A_1^\omega, \dots, A_n^\omega$ is the same as sorting $\text{root}(A_1)^\omega, \dots, \text{root}(A_n)^\omega$. Therefore, we can replace A_i by $\text{root}(A_i)$. Henceforth, $A_1, \dots, A_n$ are **primitive**.

Clearly, $\text{root}(A_i)$ can be computed in $O(|A_i|)$ time. According to Lemma 2, computing $\text{root}(A)$ reduces to finding the largest $j < |A|$ such that the prefix of A with length j equals the suffix of A with length j . Moreover, the famous KMP algorithm [2] finds this j in $O(|A|)$ time.

To proceed, we need the following two crucial lemmas.

Definition 4 For any two non-empty words S and A , denote by $\deg_A(S)$ the largest integer d such that S^d is a prefix of A . Moreover, for non-empty word S and set of non-empty words $\mathcal{A} = \{A_1, \dots, A_n\}$, denote $\deg_{\mathcal{A}}(S) = \max_j \deg_{A_j}(S)$.

In other words, if we build the trie T of $\mathcal{A}$, $S^{\deg_{\mathcal{A}}(S)}$ is the longest power of S that equals to some path of the trie T starting from its root.

For any i ($1 \leq i \leq n$), denote

$$N_i = \text{the deg}_{\mathcal{A}}(A_i) - \text{th power of } A_i \quad (5)$$

$$M_i = N_i A_i^2 = \text{the}(\deg_A(A_i) + 2) - \text{th power of } A_i \quad (6)$$

We use a nontrivial example to illustrate N_i :

Through a relaxation of Proposition 5 of [16], we have

Fig. 1 Example of Definition 4

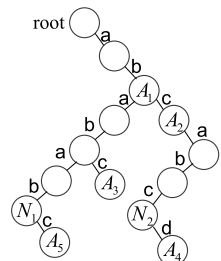
$$A_1 = ab \quad N_1 = ababab$$

$$A_2 = abc \quad N_2 = abcabc$$

$$A_3 = \text{ababc} = N_3$$

$$A_4 = \text{abcabcd} = N_4$$

$$A_5 = \text{abababc} = N_5$$



Lemma 4 If $A^\omega \neq B^\omega$, there is a mismatch between prefixes of A^ω and B^ω within the length of $|A| + |B|$.

Lemma 4 can further help us derive:

Lemma 5 The relation between A_i^ω and A_j^ω is the same as the relation between M_i and M_j , that is,

$$\begin{aligned} A_i^\omega = A_j^\omega &\Leftrightarrow M_i = M_j, \\ A_i^\omega < A_j^\omega &\Leftrightarrow M_i < M_j, \\ A_i^\omega > A_j^\omega &\Leftrightarrow M_i > M_j. \end{aligned}$$

As a corollary, sorting $A_1^\omega, \dots, A_n^\omega$ is equivalent to sorting $M_1, \dots, M_n$.

Proof Without loss of generality, we assume $A_i \neq A_j$ and $|A_i| < |A_j|$. We claim that the longest common prefix between A_i^ω and A_j^ω followed by a mismatch, must be within a prefix of M_i and M_j thus showing the lemma.

By definition 4, M_j contains at least two copies of A_j , so $|M_j| \geq 2|A_j|$. Since

$|M_j| \geq 2|A_j| > |A_i| + |A_j|$ (Recall we assume $|A_i| < |A_j|$), the comparison between A_i^ω and A_j^ω will certainly terminate within $|M_j|$.

By the definition of N_i (Definition 4), N_i is the maximum power of A_i such that N_i is a prefix of a string of $A_1, \dots, A_n$. If $N_i + A_i$ is not a prefix of A_j^ω , the mismatch occurs within

$|N_i| + |A_i|$, then we have the mismatch occurs within $|M_i| = |N_i| + 2|A_i| > |N_i| + |A_i|$. Otherwise, since $N_i + A_i$ is a prefix of A_j^ω , we have $|N_i| + |A_i| > |A_j|$ by Definition 4.

Thus $|M_i| = |N_i| + 2|A_i| > |A_i| + |A_j|$. By Lemma 4, the mismatch must clearly be within $|M_i|$. Thus, the comparison between A_i^ω and A_j^ω reduces to the comparison between

M_i and M_j . $\square$

Lemma 5 reduces comparing A_i^ω, A_j^ω to comparing their prefixes M_i, M_j . We should mention that results similar to Lemma 5 are reported in literature, e.g., in [23] and [16]. However, to the best of our knowledge, the previous results are not quite the same; they only discuss particular cases. For example, the case where A_i is a factorial power of A_j is not considered in [23] and [16].

Assume $A_1, \dots, A_n$ are **distinct** henceforth in this section. To this end, we can use a trie to reduce those duplicate elements in $A_1, \dots, A_n$.

Lemma 6 When $A_1, \dots, A_n$ are primitive and distinct, $\sum_i |N_i| = O(L)$.

Proof First, we argue that $N_1, \dots, N_n$ are distinct. Suppose that $N_i = N_j$ ($i \neq j$). Recall that $N_i = A_i^m$ (for $m = \deg_{\mathcal{A}}(A_i)$) and $N_j = A_j^n$ (for $n = \deg_{\mathcal{A}}(A_j)$). Applying Lemma 1

(condition 3), $A_i = C^k$ and $A_j = C^l$. Further since A_i, A_j are primitive, $A_i = C = A_j$, which contradicts the assumption that $A_i \neq A_j$.

We say N_i *extremal* if it is **not** a prefix of any word in $\{N_1, \dots, N_n\} \setminus \{N_i\}$. Partition $N_1, \dots, N_n$ into several groups such that (a) for elements in the same group, one of them is the prefix of the other, and (b) the longest element in each group is extremal. (It is obvious that such a partition exists: we can first distribute the extremal ones to different groups, and then distribute the non-extremal ones to suitable groups (each non-extremal one is a prefix of a certain extremal one). A graphic example is given at the end of the proof.

Now, consider any such group, e.g., $N_{i,1}, \dots, N_{i,x}$. It suffices to prove that $|N_{i,1}| + \dots + |N_{i,x}| = O(|A_{i,1}| + \dots + |A_{i,x}|)$, and we prove it in the following. Without

loss of generality, assume that $N_{i,j}$ is a prefix of $N_{i,j+1}$ for $j < x$.

We state two important formulas: (i) $N_{i,x} = A_{i,x}$. (ii) $|N_{i,j}| < |A_{i,j}| + |A_{i,j+1}|$ for $j < x$. Equation $|N_{i,1}| + \dots + |N_{i,x}| = O(|A_{i,1}| + \dots + |A_{i,x}|)$ above follows from formulas (i) and (ii) immediately.

Proof of (i). Suppose to the contrary that $N_{i,x} \neq A_{i,x}$. By the definition of $N_{i,x}$, there exists some A_j such that $N_{i,x}$ is a prefix of A_j . Clearly, $j \neq i, x$ since $N_{i,x}$ is not a prefix of $A_{i,x}$. Consequently, $N_{i,x}$ is a prefix of some other N_j , which means $N_{i,x}$ is not extremal, contradicting property (b) of the grouping mentioned above.

Proof of (ii). Suppose to the contrary that $|N_{i,j}| \geq |A_{i,j}| + |A_{i,j+1}|$. Since $N_{i,j}$ and $N_{i,j+1}$ are powers of $A_{i,j}$ and $A_{i,j+1}$ and share a common prefix, $N_{i,j}$, of length at least

$|A_{i,j}| + |A_{i,j+1}|$, by Lemma 1 (condition 2), $A_{i,j} = C^k$ and $A_{i,j+1} = C^l$ for some C, k, l .

Hence $A_{i,j} = A_{i,j+1}$, as $A_{i,j}$ and $A_{i,j+1}$ are primitive. Contradictory. $\square$

Our algorithm for sorting $A_1^\omega, \dots, A_n^\omega$ is simply as follows; see also Algorithm 1. First, build a trie of $A_1, \dots, A_n$ and use it to compute $N_1, \dots, N_n$. For computing N_i , we walk along the trie from the root and search for maximal powers of A_i , which takes $O(|N_i| + |A_i|) = O(|N_i|)$ time. The total running time for computing $N_1, \dots, N_n$ is therefore $O(\sum_i |N_i|) = O(L)$.

Second, we compute $M_1, \dots, M_n$ and build a trie of them. By utilizing this trie, we obtain the lexicographic order of $M_1, \dots, M_n$, which equals the order of $A_1^\omega, \dots, A_n^\omega$ according to Lemma 5. The running time of the second step is

$$\sum_i |M_i| = \sum_i |N_i| + 2 \sum_i |A_i| = O(L) + O(L) = O(L).$$

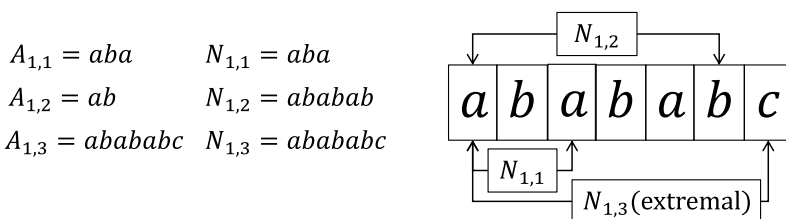


Fig. 2 Illustration of Lemma 6

Algorithm 1 Algorithm for sorting $A_1^\omega, \dots, A_n^\omega$ **Input:** Words $A_1, \dots, A_n$.**Output:** The order of $A_1^\omega, \dots, A_n^\omega$.

- 1: Build the trie of $A_1, \dots, A_n$.
- 2: **for** each $i \in \{1, \dots, n\}$ **do**
- 3: Walk down the trie from the root to fetch the maximal power of A_i .
- 4: **end for**
- 5: Compute $M_1, \dots, M_n$ and build the second trie of them.
- 6: Obtain the lexicographic order of $M_1, \dots, M_n$ utilizing the second trie.
- 7: Output lexicographic order of $M_1, \dots, M_n$, which equals to the order of $A_1^\omega, \dots, A_n^\omega$.

Step 1 takes $O(\sum_i |A_i|) = O(L)$ time. Each step 3 takes $O(|N_i| + |A_i|) = O(|N_i|)$ time and the whole loop takes $O(\sum_i |N_i|) = O(L)$ time. Step 5 takes $\sum_i |M_i| = \sum_i |N_i| + 2 \sum_i |A_i| = O(L) + O(L) = O(L)$ time. Both step 6 and 7 also

take $O(L)$ time. In total, this algorithm takes $O(L)$ time.

As a comparison, there exists a less efficient algorithm for solving Problem 1, which is based on a standard sorting algorithm associated with a naïve gadget for comparing A^ω and B^ω – according to Lemma 3, comparing A^ω and B^ω reduces to comparing AB and BA , which takes $O(|A| + |B|)$ time. The time complexity of this alternative algorithm is higher. For example, when $A_1 = \text{“aaaaaa1”}$, $A_2 = \text{“aaaaaa2”}$, etc, the running time would be $\Omega(n \log n |A_1|) = \Omega(L \log n)$.

2.1 Integer alphabet

The algorithm given above does not run in $O(L)$ time for large σ . Each node-level step takes $O(\sigma)$ time during the construction of the tries and during the search of $M_1, \dots, M_n$. Therefore, the total running time is actually $O(\sigma L)$ for arbitrary σ . Note that using balanced binary search trees (e.g. AVL trees) to manage child nodes of tries would yield $O(L \log \sigma)$ time. Nevertheless, we can improve the algorithm to $O(\sigma + L)$ time when Σ is an integer alphabet as shown below.

Assume $A_1, \dots, A_n$ are primitive (recall the statement just after Theorem 1). Moreover, remove duplicate elements in $A_1, \dots, A_n$. To this end, we can first sort $A_1, \dots, A_n$ in $O(L + \sigma)$ time using LSD radix sort [24] (section 3.2, Page 77–84), and then delete adjacent duplicate strings in $O(L)$ time. Now $A_1^\omega, \dots, A_n^\omega$ are distinct.

Let M'_i be the shortest prefix of A_i^ω that is not a prefix of A_j^ω for all $j \neq i$. (See an illustration of M'_i in Example 1). Note that $|M'_i| \leq |M_i|$, because M_i is a prefix of A_i^ω that is not a prefix of A_j^ω for all $j \neq i$ (recall that the longest common prefix between A_i^ω and A_j^ω followed

by a mismatch, must be within a prefix of M_i and M_j , as introduced in the proof of Lemma 5).

Moreover, denote $L' = \sum_{i=1}^n |M'_i| \leq \sum_{i=1}^n |M_i|$, which is $O(L)$ followed by Lemma 6.

Example 1 (An illustration of distinguishing prefix M'_i)

Consider the following four primitive and distinct words:

$$A_1 = ab, \quad A_2 = aba, \quad A_3 = abaab, \quad A_4 = ababc.$$

The distinguishing prefix M'_i is underlined in the corresponding repeating string as follows.

$$\begin{aligned} A_1^\omega &= \underline{ab}ababab \cdots, \\ A_2^\omega &= \underline{aba}abaaba \cdots, \\ A_3^\omega &= \underline{abaab}abaababaab \cdots, \\ A_4^\omega &= \underline{ababc}ababcababc \cdots. \end{aligned}$$

Note that we show each repeating string A_i^ω with M_i as the prefix (which is before symbol "..."). It is easy to see that M'_i is a prefix of M_i .

Suppose for the time being that $M'_1, \dots, M'_n$ have been computed. We then sort them in

$O(L' + \sigma) = O(L + \sigma)$ time using the LSD radix sort [24]. According to the definition of M'_i 's, the sorting result of $M'_1, \dots, M'_n$ is the same as that of $A_1^\omega, \dots, A_n^\omega$. Therefore, it only remains to show how to compute $M'_1, \dots, M'_n$.

We can compute $M'_1, \dots, M'_n$ in $O(L' + \sigma)$ time using the algorithm given by Paige and Tarjan [25] (Alg-PT). In the original paper, Alg-PT assumes that $W_1, \dots, W_n$ are finite, and it computes $M'_1, \dots, M'_n$ in $O(\sum_{i=1}^n |M'_i|)$ time, where M'_i denotes the shortest prefix of

W_i that is not a prefix of W_j for all $j \neq i$. Nevertheless, Alg-PT can be adapted to our case where $W_i = A_i^\omega$ is infinite. We describe this original algorithm in the following, and leave the easy adaption to infinite case to the readers.

The general idea of Alg-PT is simple. Initially, it puts all n strings into one big group (called the 1st group), sharing the empty string as their common prefix. When a group (all starting with a common prefix S) has more than one element, alg-PT partitions this group into smaller groups according to their $(|S| + 1)$ -th characters (but **not** necessarily in lexicographic order). Finally, each group contains only one element (say W_i) with common prefix S , and S is exactly the prefix M'_i we are looking for.

This partitioning process can be handled by a BFS as follows. List L_g stores those i such that W_i is in the g -th group; for example, $L_1 = \{1, \dots, n\}$. Integer len_g is the length of the confirmed common prefix S shared by the g -th group; for example, $\text{len}_1 = 0$. Moreover, queue Q stores the indices of all the groups to be partitioned; initially, Q contains only 1. In each step, we take out an index g from the queue. If L_g contains only one element i , we determine that M'_i equals the prefix of A_i with length len_g . Otherwise (L_g has more than one elements)

we partition L_g in three steps: 1. Throw every $i \in L_g$ into $\text{Bucket}[A_i[\text{len}_g + 1]]$ (note that the

elements in any bucket can be stored by a chain), and at the same time maintain a list B of all nonempty buckets. 2. For each $c \in B$ (i.e., $\text{Bucket}[c] \neq \emptyset$), create a new group g' and push it to Q . $L_{g'} = \text{Bucket}[c]$, whereas $\text{len}_{g'} = \text{len}_g + 1$. 3. Clear all the nonempty buckets using B .

Running time analysis. There are σ buckets, and initializing them costs $O(\sigma)$ time. Each partition costs only $O(|L_g|)$ time. Since each string W_i contributes to at most $|M'_i|$ partitions, the total time for partitioning across all groups is $O(\sum_{i=1}^n |M'_i|)$.

As a summary, we obtain the following result.

Theorem 2 $A_1^\omega, \dots, A_n^\omega$ can be sorted in $O(L + \sigma)$ time when Σ is an integer alphabet.

For linearly sortable alphabets (i.e., $\sigma = L^{O(1)}$), $A_1^\omega, \dots, A_n^\omega$ can be sorted in $O(L)$

time. (When $\sigma = L^{O(1)}$, we can make $\sigma = O(L)$ with an $O(L)$ time preprocessing.)

2.2 Comparison based model

In this subsection, we restrict ourselves to a model where we are allowed to compare two symbols in Σ but not read or manipulate a single symbol. For this model, we will give an algorithm that runs in $O(L + n \log n)$ time and the main ingredient of this algorithm is the Three-Way Radix Quicksort [1, 26] and ternary trie [1].

Three-Way Radix Quicksort sorts n strings of total length L in time $O(n \log n + L)$. It works by partitioning the set of strings according to the first symbol, then recursively sorting the three groups: strings whose first character is less than the pivot, equal to the pivot, and greater than the pivot. The first group and third group are, respectively, sorted again according to the first symbol and the second group is refined based on the next symbol. A ternary trie is a trie where each node has three children: left (for smaller symbols), middle (for equal symbols), and right (for larger symbols). In fact, the ternary trie can be seen as the recursive structure induced by Three-Way Radix Quicksort. We remark that the $O(n \log n + L)$ running time bound is guaranteed in the worst case. This is because we can partition around the median (which can be found in linear time) of the elements to get a perfectly balanced partition in each round of Three-Way Radix Quicksort or the building process of ternary trie [1, 26].

Theorem 3 $A_1^\omega, \dots, A_n^\omega$ can be sorted in $O(L + n \log n)$ time under the comparison based model.

Proof The algorithm is similar to Algorithm 1.

Step 1. We sort $A_1, \dots, A_n$. But we use Three-Way Radix Quicksort [1] this time, which only requires symbol comparison. It takes $O(L + n \log n)$ time.

Step 2. We use the ternary trie of $A_1, \dots, A_n$ to compute $M_1, \dots, M_n$. Notice that it takes $O(\log n + |M_i|)$ time to compute M_i . So computing $M_1, \dots, M_n$ costs $O(L + n \log n)$ time as well (applying Lemma 6).

Step 3. In another $O(L + n \log n)$ time, we sort $M_1, \dots, M_n$ by Three-Way Radix Quicksort to obtain the order of $A_1^\omega, \dots, A_n^\omega$ (applying Lemma 5). $\square$

Remark 1 We remark that the time complexity of our algorithms for sorting $A_1^\omega, \dots, A_n^\omega$ under each model (bounded alphabet, unbounded alphabet, comparison based model) are aligned with those for sorting $A_1, \dots, A_n$.

2.3 Comparison with extended Burrows-Wheeler Transform

As is mentioned in Section 1, extended Burrows-Wheeler Transform [16] sorts all conjugates of the given words in ω -order. In this section, we briefly look into the latest eBWT algorithm [20] called SAIS-for-eBWT and discuss the difference between the algorithm in Section 2.1 and SAIS-for-eBWT.

In SAIS-for-eBWT, given $\mathcal{M} = A_1, \dots, A_n$, the algorithm returns the generalized suffix array $(A, \mathcal{I})$ of $\mathcal{M}$, where A is the sorted list of all conjugates (positions) of $\mathcal{M}$ in ω -order and $\mathcal{I}$ is the set of the indices in A which mark positions of the words originally given. (The resulting $A[i] = (j, d)$ indicates that the conjugate starting from $A_d[j]$ is the i -th among all conjugates in ω -order.)

The algorithm classifies each position as cyclic L-type or S-type in linear time (Corollary 7 of [20]). An LMS position is an S-type position preceded by an L-type position. LMS-order and LMS-substrings in [20], adapted from [19] to cyclic strings, are defined over all conjugates of $\mathcal{M}$. Note that LMS-order in [20] is a refinement of ω -order: $U <_{\text{LMS}} V$ implies $U^\omega < V^\omega$, but not vice versa. See details of the definitions in [20].

In SAIS-for-eBWT, a critical procedure adapted from [19] is called Induced Sorting. See Algorithm 2 for structured description.

Algorithm 2 Induced Sorting [20]

-
- 1: Insert all LMS-positions at the end of their respective buckets; initialize $head(b)$, $tail(b)$ for all buckets b
 - 2: Induce L-type (left-to-right): from (j, d) at $A[i]$, put $(j - 1, d)$ (if L-type) at head of bucket $T_d[j - 1]$; advance the head
 - 3: Induce S-type (right-to-left): from (j, d) at $A[i]$, put $(j - 1, d)$ (if S-type) at tail of bucket $T_d[j - 1]$; retreat the tail
-

The Induced Sorting procedure performs linear-time traversals to sort all conjugates in LMS-order, but it does not guarantee to sort them in ω -order unless LMS positions are inputted in ω -order in the first step.

SAIS-for-eBWT first employs the Induced Sorting procedure [19] to sort LMS-substrings in LMS-order. Then the algorithm sorts all conjugates starting from the LMS positions in ω -order recursively. Given LMS positions sorted in ω -order, the Induced Sorting procedure then sorts all the positions of $\mathcal{M}$ in ω -order. See Algorithm 3 for the whole algorithm.

Algorithm 3 SAIS-for-eBWT($\mathcal{M}$) [20]

-
- 1: Remove strings of length 1 from $\mathcal{M}$.
 - 2: Assign cyclic types to all positions of strings from $\mathcal{M}$. $\triangleright O(L)$
 - 3: Use procedure Induced Sorting to sort LMS-substrings. $\triangleright O(L)$
 - 4: Assign names to cyclic LMS-substrings;
 - 5: **if** the names are distinct **then**
 - 6: Replace the LMS-substrings in $\mathcal{M}$ with the names to obtain $\mathcal{M}_0$.
 - 7: SAIS-for-eBWT($\mathcal{M}_0$), returning array A_0 , map A_0 back to A . $\triangleright T(\frac{L}{2})$
 - 8: **end if**
 - 9: Use procedure Induced Sorting to sort all positions in $\mathcal{M}$, add length-1 strings in their respective positions, return $(A, \mathcal{I})$. $\triangleright O(L)$
-

In summary, SAIS-for-eBWT is recursive, utilizing linear-time traversals before and after each recursive call. Since the problem size is halved at each recursion, the overall time complexity remains linear. The algorithm is recursive and involves complicated definitions such as L/S/LMS and sub-procedures like Induced Sorting.

In comparison, the linear time algorithm of Section 2.1 can sort all the given words in ω -order and involves basic definitions and sorting algorithms without recursive calls. If the goal is to solve Problem 1 for the smallest rearrangement, then only the ω -order of the given words are needed, not the ω -order of all conjugates. So the trade-off for simplicity is achieved in our result.

3 The string rearrangement inequalities

We call equation (7) right below *the String Rearrangement Inequalities*.

Theorem 4 For non-empty words $A_1, \dots, A_n$, where $A_1^\omega \leq \dots \leq A_n^\omega$, we claim that

$$A_1 A_2 \cdots A_n \leq A_{\pi_1} A_{\pi_2} \cdots A_{\pi_n} \leq A_n A_{n-1} \cdots A_1, \quad (7)$$

for any permutation $(\pi_1, \dots, \pi_n)$ of $\{1, \dots, n\}$.

In other words, if several words are to be rearranged and concatenated into a string S , the lexicographically smallest outcome of S occurs when the words are arranged so that they are increasing in ω -order, and the lexicographically largest outcome of S occurs when the words are arranged so that they are decreasing in ω -order.

Example 2 Suppose there are four given words: “123”, “12”, “121”, “1212”. Notice that

$(121)^\omega < (12)^\omega = (1212)^\omega < (123)^\omega$. Applying Theorem 4, the lexicographically smallest

outcome would be “121121212123”, and the lexicographically largest outcome would be “123121212121”.

Proof of Theorem 4 Consider any concatenation $A_{\pi_1} \cdots A_{\pi_n}$. If A_1 is not at the leftmost position, we swap it with its left neighbor A_x . Note that $A_1^\omega \leq A_x^\omega$ by assumption. According to Lemma 3, $A_1 A_x \leq A_x A_1$. This means that the entire string becomes smaller or remains unchanged after the swapping. Applying several such swappings, A_1 will be on the leftmost position. Then, we swap A_2 to the second place. So on and so forth. It follows that $A_1 \cdots A_n \leq A_{\pi_1} \cdots A_{\pi_n}$.

The other inequality in (7) can be proved symmetrically; proof omitted. $\square$

Combining Theorem 4 with Theorem 2, we obtain

Corollary 1 For linearly sortable alphabets, given n words $A_1, \dots, A_n$ that are to be rearranged and concatenated, the smallest and largest concatenation can be found in $O(\sum_i |A_i|) = O(L)$ time.

4 More applications

Without loss of generality, assume that $A_1^\omega \leq \cdots \leq A_n^\omega$ in this section. If not, we can sort the given words, which will not affect the time bound of the algorithms below. To implement Theorem 2, we assume the alphabet Σ is linearly sortable in this section.

4.1 Rearrangement with rotation problem

Circular strings such as DNA sequences of bacteria lack a predefined origin, making their representations equivalent under rotation. Computing the lexicographically minimal rotation provides a canonical form essential for comparing circular data structures [7, 27, 28]. It is reasonable to generalize the problem to the combinatorial problem: given n words, rearrange and concatenate them such that the minimum rotation of the resulting circular string is lexicographically smallest. The problem is formally written as follows.

Problem 2. Given n non-empty words $A_1, \dots, A_n$ over alphabet Σ , rearrange and concatenate $A_1, \dots, A_n$ so that the minimum representation (i.e. minimum rotation) of obtained string S is lexicographically smallest. Equivalently, we ask for the minimum of all rotations of all rearranged concatenations of $A_1, \dots, A_n$.

Let $S[a : b]$ denote $S[a] \cdots S[b]$. The problem asks for

$$S^* = \min_{(\pi_1, \dots, \pi_n)} \min_i (S[i : L-1]S[0 : i-1] \mid S = A_{\pi_1} \cdots A_{\pi_n})$$

Clearly, S^* must start with a part of a given word, say A_d ($d \in \{1, \dots, n\}$ and d is unknown). In other words, $S^* = qA_{\pi_1} \cdots A_{\pi_{n-1}}p$ where $A_d = pq$ (q is nonempty) for some d . Moreover, we know that (i) the words except A_d in S^* must be increasing in ω -order, that is,

$$S^* = qA_1 \cdots A_{d-1}A_{d+1} \cdots A_n p \quad (pq = A_d, q \neq \epsilon)$$

Proof of (i). Suppose to the contrary that the words except A_d in S^* are not increasing in ω -order. Keep the beginning q and the end p and rearrange the words in between so that they are increasing in ω -order. By Theorem 4, the result is smaller, hence S^* is not optimal, contradictory.

Now, for each $k \in \{1, \dots, n\}$, denote by S_k the smallest string among all strings of the form $qA_1 \cdots A_{k-1}A_{k+1} \cdots A_n p$ where $pq = A_k$ and $q \neq \epsilon$, and call S_k a *candidate*. An approach for computing S^* would be as follows: compute all candidates and select the best among them.

We generally adopt this approach but with some necessary changes: We do not compute $S_1, \dots, S_n$ precisely as that would be time consuming. Instead, we compute n *backup-*

candidates $S'_1, \dots, S'_n$ (which are easier to compute) and select the smallest among the backup-candidates.

Formally, S'_k is the string obtained by Algorithm 4, which is still of the form $qA_1 \cdots A_{k-1}A_{k+1} \cdots A_n p$ where $pq = A_k$ and $q \neq \epsilon$, but may not be the smallest one (S_k) of this form. Nonetheless, it holds that $S'_k = S_k$ for the case $k = d$. Consequently, the smallest among $S'_1, \dots, S'_n$ still equals S^* . (To see this, notice that $S^* = S_d = S'_d$ and $S^* \leq S'_k$.)

Strings of the form $qA_1 \cdots A_{k-1}A_{k+1} \cdots A_n p$ can be represented by a pair (k, i) where $p = A_k[0 : i - 1]$, $q = A_k[i : |A_k| - 1]$, and i is called the *split position*.

Lemma 7 *We can compute the split position of the backup-candidate S'_k in $O(|A_k|)$ time, and hence compute $S'_1, \dots, S'_n$ in $O(L)$ time.*

Lemma 8 *Given the split positions of $S'_1, \dots, S'_n$ and the concatenated string $S = A_1 \cdots A_n$, we can find S^* (the smallest among $S'_1, \dots, S'_n$) in $O(n)$ time using $O(1)$ time Longest Common Extension (LCE) queries – return the length of the longest common prefix of two suffixes.*

Combining the last two lemmas (proved below) with the well-known fact that the LCE queries can be answered in $O(1)$ time after $O(L)$ (linear) time preprocessing on S (Lemma 9), the running time of our algorithm is $O(L)$.

Theorem 5 *We can solve Problem 2 in $O(L)$ time for linearly sortable alphabets (after sorting $A_1, \dots, A_n$ in ω -order so that $A_1^\omega \leq \dots \leq A_n^\omega$).*

Algorithm 4 Algorithm for computing backup-candidate S'_k

Input: Words $A_1, \dots, A_n$ with $A_1^\omega \leq \dots \leq A_n^\omega$, index k

Output: The split position i where $p = A_k[0 : i - 1]$, $q = A_k[i : |A_k| - 1]$ and $S'_k = qA_1 \dots A_{k-1}A_{k+1} \dots A_np$.

```

1: Denote  $T = A_kA_1 \dots A_{k-1}A_{k+1} \dots A_n$ ;
2:  $i = 0, j = 1, l = 0$ ;
3: while  $i + l \leq |A_k|$  do
4:   if  $T[i + l] < T[j + l]$  then
5:      $j = j + l + 1, l = 0$ ;
6:   else if  $T[i + l] > T[j + l]$  then
7:      $i = i + l + 1$ ;
8:     if  $i > j$  then
9:        $j = i + 1, l = 0$ ;
10:    else
11:       $i = j, j = j + 1, l = 0$ ;
12:    end if
13:  end if
14:  while  $T[i + l] = T[j + l]$  and  $i + l \leq |A_k|$  do
15:     $l = l + 1$ ;
16:  end while
17: end while
18:  $q = T[i : |A_k| - 1], p = T[0 : i - 1]$ ;
19: return  $i$ .
```

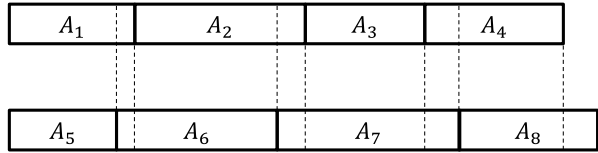
Proof of Lemma 7 Denote $T = A_kA_1 \dots A_{k-1}A_{k+1} \dots A_n$. Algorithm 4 roughly aims at computing the smallest string S_k of the form $qA_1 \dots A_{k-1}A_{k+1} \dots A_np$ where $pq = A_k$ and $q \neq \epsilon$. But instead of finding the smallest string (denoted as S_k) of this form, it may actually find an alternative S'_k .

This algorithm is a variant of the classic algorithm of Shiloach [29] for finding the smallest rotation of a given string. Shiloach's algorithm maintains two candidate starting indices i and j (initially 0 and 1, we assume $i < j$ otherwise swap them) and a matching length l (initially 0). At each step, it compares characters $T[i + l]$ and $T[j + l]$. If they are equal, l is incremented; otherwise all l candidate starting indices starting from the candidate with the larger character can be safely skipped (we call them dead) and l is reset. The algorithm keeps indices in $\{1, \dots, j - 1\} \setminus i$ dead and advances the candidate starting indices over all dead indices. The algorithm terminates when j exceeds the length of T , and the i gives the smallest rotation.

Fig. 3 An example for comparison between S'_a and S'_b . Note that some U_i and V_i may be empty in general cases

q_a		$A_1 \dots A_{a-1}$		$A_{a+1} \dots A_n$	p_a	
U_1	U_2	U_3	U_4	U_5	U_6	U_7
V_1	V_2	V_3	V_4	V_5	V_6	V_7
q_b		$A_1 \dots A_{b-1}$		$A_{b+1} \dots A_n$	p_b	

Fig. 4 An example for comparison between $A_1 \cdots A_4$ and $A_5 \cdots A_8$



Compared with his original algorithm, we modify step 3 and the latter part of step 14 – when $i + l$ reaches out of $|A_k|$, we stop the algorithm immediately and output the rotation of T starting with $T[i]$ (denoted as S'_k). The reason that we can adopt this early-stop mechanism is as follows.

(1) If j beats i somewhere further (i.e. $T[j + l] < T[i + l]$ for some $i + l > |A_k|$), the best rotation will reach out of $|A_k|$, which means the optimal solution S^* is not S_k . It is allowed to select some $S'_k \neq S_k$ in this case.

(2) Otherwise, i is the optimal option. Therefore we make sure that $S'_k = S_k$ in this case.

By the early-stop mechanism, the number of steps for computing S'_k is $O(|A_k|)$, not $O(|T|) = O(L)$. The whole process of finding $S'_1, \dots, S'_n$ is $O(\sum_{k=1}^n |A_k|) = O(L)$. $\square$

Proof of Lemma 8 We prove that using $O(1)$ time LCE queries, we can compare the lexicographic order of strings S'_a and S'_b in $O(1)$ time, and thus compute S^* in $O(n)$ time.

For $k \in \{a, b\}$, cut S'_k into 4 parts, q_k , $A_1 \cdots A_{k-1}$, $A_{k+1} \cdots A_n$ and p_k (where $A_k = pq$), each of which is a substring of S . According to these cutting lines, S'_a and S'_b are respectively divided into 7 parts (see Fig. 3), denoted by $U_1, \dots, U_7$, $V_1, \dots, V_7$ in the following. Comparing S'_a, S'_b reduces to comparing U_i, V_i for $i = 1, \dots, 7$. Since U_i and V_i are substrings of S , comparing U_i, V_i only takes $O(1)$ time using the longest common extension (LCE) queries on S . $\square$

Lemma 9 ([30]) *For a given string S , Longest Common Extension (LCE) queries, which return the length of the longest common prefix of two suffixes of S , can be answered in $O(1)$ time per query after $O(|S|)$ (linear) time preprocessing.*

Proof One approach for the $O(1)$ LCE query is as follows: (1) First, construct the suffix array (SA) of S in $O(n)$ time, which orders all suffixes of S lexicographically; (2) Compute the longest common prefix (LCP) array (of length $|S|$) for the suffix array, where $\text{LCP}[i]$ denotes the length of the longest common prefix between the i -th and $(i - 1)$ -th suffixes in the SA; (3) Preprocess the LCP array with a range minimum query (RMQ) data structure (e.g., a sparse table) in $O(n)$ time, which supports $O(1)$ time minimum queries over any interval of the LCP array.

For any LCE query on suffixes starting at positions i and j of S , the length of the longest common extension can be computed as the minimum value of the LCP array over the interval corresponding to the positions of these two suffixes in the SA, a query that the RMQ

structure answers in $O(1)$ time. Full details of the construction and query logic are given in [30]. $\square$

4.2 k -rearrangement problem

In the field of computational biology and bioinformatics, resource constraints often require selecting k representative items from n available candidates [31–33]. These selection methods optimize criteria such as cost function [31] or average distance [32], yet treat the chosen subset as unordered. Combined with our result of rearrangement, the selected k items can be rearranged in a canonical order for standardized storage. This leads to a combinatorial question:

Instead of concatenating all n words, how to select k words from the given n words so that the concatenation is smallest. The problem is formally written as follows.

Problem 3. Given non-empty words $A_1, \dots, A_n$ over alphabet Σ , select $k \leq n$ words $A_{\pi_1}, \dots, A_{\pi_k}$, where $\pi_1, \dots, \pi_k$ are distinct indices, so that $S = A_{\pi_1} \cdots A_{\pi_k}$ is lexicographically smallest.

Recall that $A_1^\omega \leq \dots \leq A_n^\omega$.

We solve Problem 3 by dynamic programming. Let $\text{suf}[i][j]$ be the minimum string rearranged and concatenated by j strings chosen from $A_i, \dots, A_n$. The answer to the problem is $\text{suf}[1][k]$ by definition. In terms of boundary conditions, $\text{suf}[i][1]$ is the smallest A_j in lexicographic order where $i \leq j \leq n$. Moreover, the transition equation for computing $\text{suf}[i][j]$ is as follows.

$$\text{suf}[i][j] = \min(\text{suf}[i+1][j], A_i + \text{suf}[i+1][j-1])$$

where $\min$ denotes lexicographic minimum and $+$ denotes string concatenation.

During each state transition, we need to perform a comparison between two states, $\text{suf}[i+1][j]$ and $A_i + \text{suf}[i+1][j-1]$. Using the technique similar to subsection 4.1, we divide each of them into $O(j) = O(k)$ substrings. The divided substrings of $\text{suf}[i+1][j]$ and $A_i + \text{suf}[i+1][j-1]$ are aligned (Fig. 4). Using $O(1)$ time longest common extension (LCE) queries after $O(L)$ (linear) time preprocessing on $A_1 \cdots A_n$ (Lemma 9), we can

compare in $O(k)$ time. The total running time is $O(nk) \times O(k) + O(L) = O(nk^2 + L)$.

5 Conclusions

In this paper, we present a linear time algorithm for sorting the strings $A_1, \dots, A_n$ so that $A_1^\omega \leq \dots \leq A_n^\omega$, which beats the trivial algorithm by a factor of $\log n$. Our algorithm itself is direct (indeed, it looks somewhat brute-force) and easy to implement, yet the analysis of its correctness and complexity is built upon nontrivial observations, namely, Lemma 5, and Lemma 6. By implementing the algorithm along with the string rearrangement inequalities, we solve several rearranging problems efficiently.

In the future, it is a problem worth attacking that whether we can improve the running time for sorting $A_1^\omega, \dots, A_n^\omega$ from $O(L)$ to $O(N)$ when given the trie of $A_1, \dots, A_n$, where N denotes the number of nodes in the trie of $A_1, \dots, A_n$.

A proof of Lemma 2

Proof *Fact 1.* If $S = BB' = B'B$ and B, B' are non-empty, S is non-primitive.

This is implied by Lemma 1 (condition 1); proof omitted.

Claim 1. $|\text{root}(A)| \geq k$.

Proof: The prefix and suffix of A with length $|A| - |\text{root}(A)|$ are the same, which implies that $j \geq |A| - |\text{root}(A)|$. Consequently, $|\text{root}(A)| \geq |A| - j = k$.

Claim 2. If $|\text{root}(A)| < |A|$ (i.e., A is non-primitive), then $k \geq |\text{root}(A)|$.

Proof: Denote $S = \text{root}(A)$ and assume $|S| < |A|$. Therefore, $A = S^d$ ($d \geq 2$). Suppose to the opposite that $k < |\text{root}(A)|$. Let B be the prefix of S with length k , and B' be

the suffix of S such that $S = BB'$. As $k < |S|$, we have $j > |A| - |S| \geq |S|$. Further since the suffix of A with length j (which starts with $B'B$) equals to the prefix of A with length j (which starts with $S = BB'$), we get $S = B'B$. Applying Fact 1, $\text{root}(A) = S$ is non-primitive. Contradictory.

We are ready to prove the lemma. When $|A|$ is a multiple of k , A is a power of its prefix of length k , which means $|\text{root}(A)| \leq k$. Further by Claim 1, $|\text{root}(A)| = k$. Next, assume $|A|$ is not a multiple of k . Since $|A|$ is a multiple of $|\text{root}(A)|$, we see $|\text{root}(A)| \neq k$. Further by

Claims 1 and 2, it follows that $|\text{root}(A)| = |A|$. $\square$

A proof of Lemma 3

Proof We prove the three equivalences by considering the relationship between the finite concatenations and the infinite periodic extensions.

Case 1: $AB = BA \Leftrightarrow A^\omega = B^\omega$. If $A^\omega = B^\omega$, then $AB^\omega = AA^\omega = BB^\omega = BA^\omega$. As a result, $AB = BA$.

Conversely, assume $AB = BA$. By Lemma 1, there exists a word C and integers $k, l \geq 1$ such that $A = C^k$ and $B = C^l$. Consequently, $A^\omega = C^\omega = B^\omega$.

Case 2: $AB < BA \Leftrightarrow A^\omega < B^\omega$. We prove both directions.

$(\Rightarrow)$ Assume $AB < BA$. Without loss of generality, assume $|A| \leq |B|$ (the case

$|A| > |B|$ is symmetric). Let $k \geq 1$ be the largest integer such that $|A^k| \leq |B|$, and let A' be the prefix of A (possibly empty) satisfying $|A^k A'| = |B|$.

If $A^k A' = B$, then we have:

$$AB = A^{k+1} A' \quad \text{and} \quad BA = A^k A' A = BA.$$

Since A' is a prefix of A , the word $AB = A^{k+1}A'$ is a prefix of A^ω . Similarly, $BA = A^kA'A$ is a prefix of $B^\omega = (A^kA')^\omega$. Since $AB < BA$, a mismatch between A^ω and B^ω occurs within the first $|A| + |B|$ characters, hence $A^\omega < B^\omega$.

If $A^kA' \neq B$, then A^kA' and B differ at some position $m \leq |B|$. Since A^ω begins with A^kA' (followed by the remaining suffix of A) and B^ω begins with B , the mismatch between A^ω and B^ω occurs at the same position m , with the same ordering, hence $A^\omega < B^\omega$.

($\Leftarrow$) Assume $A^\omega < B^\omega$. The derivation of $AB < BA$ follows symmetrically from the proof of ($\Rightarrow$) by reversing the roles of the finite and infinite comparisons.

Case 3: $AB > BA \Leftrightarrow A^\omega > B^\omega$. This follows immediately by symmetry from Case 2. $\square$

Author Contributions Author Contributions: Conceptualization, R.L.; methodology, R.L., T.Z. and K.J.; formal analysis, R.L., T.Z.; writing—original draft preparation R.L. and T.Z.; writing—review and editing, K.J.; project administration, K.J.; funding acquisition, K.J. R.L. and T.Z. contributed equally to this work. All authors have read and agreed to the published version of the manuscript.

Funding This research was funded by Department of Science and Technology of Guangdong Province (Project No. 2021QN02X239), National Natural Science Foundation of China (No. 62002394), and Shenzhen Science and Technology Program (Grant No. 202206193000001, 20220817175048002).

Data Availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Bentley, J.L., Sedgewick, R.: Fast algorithms for sorting and searching strings. In: Proceedings of the Eighth Annual ACM-SIAM Symposium on Discrete Algorithms, pp. 360–369 (1997)
2. Knuth, D.E., Pratt, J.H.M.: Fast pattern matching in strings. SIAM Journal on Computing 6(2), 323–350 (1977). <https://doi.org/10.1137/0206024>
3. Lothaire, M.: Combinatorics on Words. Encyclopedia of Mathematics, Vol. 17. Addison-Wesley, MA (1983)
4. Zhang, D., Jin, K.: Fast algorithms for computing the statistics of pattern matching. IEEE Access 9, 114965–114976 (2021). <https://doi.org/10.1109/ACCESS.2021.3105607>

5. Bannai, H., I, T., Inenaga, S., Nakashima, Y., Takeda, M., Tsuruta, K.: The “runs” theorem. *SIAM Journal on Computing* **46**(5), 1501–1514 (2017). <https://doi.org/10.1137/15M1011032>
6. Crochemore, M., Russo, L.M.S.: Cartesian and lyndon trees. *Theoret. Comput. Sci.* **806**, 1–9 (2020). <https://doi.org/10.1016/j.tcs.2018.08.011>
7. Duval, J.P.: Factorizing words over an ordered alphabet. *J. Algorithms* **4**(4), 363–381 (1983). [https://doi.org/10.1016/0196-6774\(83\)90017-2](https://doi.org/10.1016/0196-6774(83)90017-2)
8. Petersen, H.: On the language of primitive words. *Theoret. Comput. Sci.* **161**, 141–156 (1996)
9. Dömösi, P., Horváth, S., Ito, M.: On the connection between formal languages and primitive words. In: *Proc. First Session on Scientific Communication*, Univ. of Oradea, Oradea, Romania, pp. 59–67 (1991)
10. Dömösi, P., Ito, M.: Context-Free Languages And Primitive Words, (2014). <https://api.semanticscholar.org/CorpusID:118541294>
11. Lischke, G.: Primitive words and roots of words. *Acta Univ. Sapientiae, Informatica* **3**(1), 5–34 (2011)
12. Berstel, J., Boasson, L.: The set of Lyndon words is not context-free. *Bull. EATCS* **63** (1997)
13. Fredricksen, H., Maiorana, J.: Necklaces of beads in k colors and k -ary de Bruijn sequences. *Discrete Math.* **23**, 207–210 (1979)
14. Au, Y.H.: Generalized de Bruijn words for primitive words and powers. *Discret. Math.* **338**(12), 2320–2331 (2015). <https://doi.org/10.1016/j.disc.2015.05.025>
15. Dolce, F., Restivo, A., Reutenauer, C.: Some variations on Lyndon words. [arXiv:1904.00954](https://arxiv.org/abs/1904.00954) (2019)
16. Mantaci, S., Restivo, A., Rosone, G., Sciortino, M.: An extension of the burrows-wheeler transform. *Theoret. Comput. Sci.* **387**(3), 298–312 (2007)
17. Kärkkäinen, J., Sanders, P.: Simple linear work suffix array construction. In: *Automata, Languages and Programming: 30th International Colloquium, ICALP 2003 Eindhoven, The Netherlands, June 30–July 4, 2003 Proceedings* 30, pp. 943–955 (2003). Springer
18. Hon, W.K., Ku, T.H., Lu, C.H., Shah, R., Thankachan, S.V.: Efficient algorithm for circular Burrows-Wheeler transform. In: *Annual Symposium on Combinatorial Pattern Matching*, pp. 257–268 (2012). Springer
19. Nong, G., Zhang, S., Chan, W.H.: Two efficient algorithms for linear time suffix array construction. *IEEE Trans. Comput.* **60**(10), 1471–1484 (2010)
20. Boucher, C., Cenzato, D., Lipták, Z., Rossi, M., Sciortino, M.: Computing the original eBWT faster, simpler, and with less memory. In: *International Symposium on String Processing and Information Retrieval*, pp. 129–142 (2021). Springer
21. Bannai, H., Kärkkäinen, J., Köppl, D., Piatkowski, M.: Constructing and indexing the bijective and extended Burrows-Wheeler transform. *Inf. Comput.* **297**, 105153 (2024)
22. Lyndon, R.C., Schützenberger, M.P.: The equation $a^m = b^n c^p$ in a free group. *Michigan Math. J.* **9**(4), 289–298 (1962). <https://doi.org/10.1307/mmj/1028998766>
23. Dolce, F., Restivo, A., Reutenauer, C.: On generalized Lyndon words. [arXiv: 1812.04515](https://arxiv.org/abs/1812.04515) (2019b)
24. Aho, A.V., Hopcroft, J.E.: *The Design and Analysis of Computer Algorithms*, 1st edn. Addison-Wesley Longman Publishing Co., Inc, USA (1974)
25. Paige, R., Tarjan, R.E.: Three partition refinement algorithms. *SIAM J. Comput.* **16**(6), 973–989 (1987). <https://doi.org/10.1137/0216062>
26. Fagerberg, R.: String sorting. In: Kao, M.Y. (ed.) *Encyclopedia of Algorithms*, pp. 907–910. Springer, Boston, MA (2008). https://doi.org/10.1007/978-0-387-30162-4_408
27. Booth, K.S.: Lexicographically least circular substrings. *Inf. Process. Lett.* **10**(4–5), 240–242 (1980)
28. Wang, Q., Ying, M.: Quantum algorithm for lexicographically minimal string rotation. *Theo. Comput. Syst.* **68**(1), 29–74 (2024)
29. Shiloach, Y.: Fast canonization of circular strings. *J. Algorithms* **2**(2), 107–121 (1981). [https://doi.org/10.1016/0196-6774\(81\)90013-4](https://doi.org/10.1016/0196-6774(81)90013-4)
30. Fischer, J., Heun, V.: Theoretical and practical improvements on the rmq-problem, with applications to lca and lce. In: Lewenstein, M., Valiente, G. (eds.) *Combinatorial Pattern Matching*, pp. 36–48. Springer, Berlin, Heidelberg (2006)
31. Somervuo, P., Koskinen, P., Mei, P., Holm, L., Auvinen, P., Paulin, L.: Barcosel: a tool for selecting an optimal barcode set for high-throughput sequencing. *BMC Bioinformatics* **19**(1), 257 (2018)
32. Kang, J.T., Zhang, P., Zöllner, S., Rosenberg, N.A.: Choosing subsamples for sequencing studies by minimizing the average distance to the closest leaf. *Genetics* **201**(2), 499–511 (2015)
33. Hawkins, J.A., Jones, S.K., Jr., Finkelstein, I.J., Press, W.H.: Indel-correcting DNA barcodes for high-throughput sequencing. *Proceedings of the National Academy of Sciences* **115**(27), 6217–6226 (2018)